WEB PLATFORM

JavaScript Runtime Visual Guide

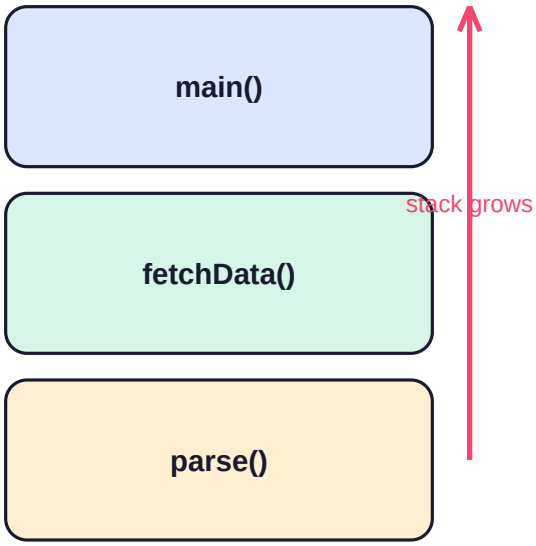
The event loop, task queues, closures,
and the prototype chain — JS under the hood.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

The Call Stack: One Thread

JS runs one thing at a time. Frames push on call, pop on return.



STACK OVERFLOW

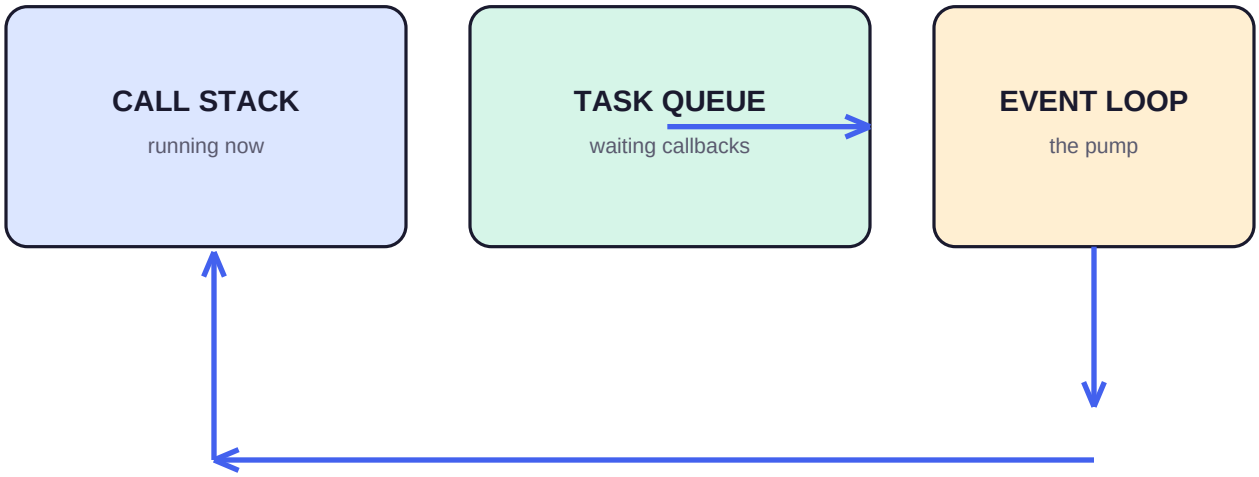
unbounded recursion — too many frames, the stack blows

Long synchronous work blocks EVERYTHING: rendering, input, timers. Never block the stack.

STEP 2/6

The Event Loop

When the stack empties, the loop pulls the next task from the queue.



Loop: stack empty? -> take oldest task -> run to completion -> repeat.

Tasks run to COMPLETION — no preemption. A 2-second task freezes the page for 2 seconds.

STEP 3/6

Microtasks vs Macrotasks

Two queues, strict priority: drain ALL microtasks before the next macrotask.

MICROTASKS

promise.then, queueMicrotask — run ASAP

MACROTASKS

setTimeout, I/O, UI events — one per loop turn

ORDER

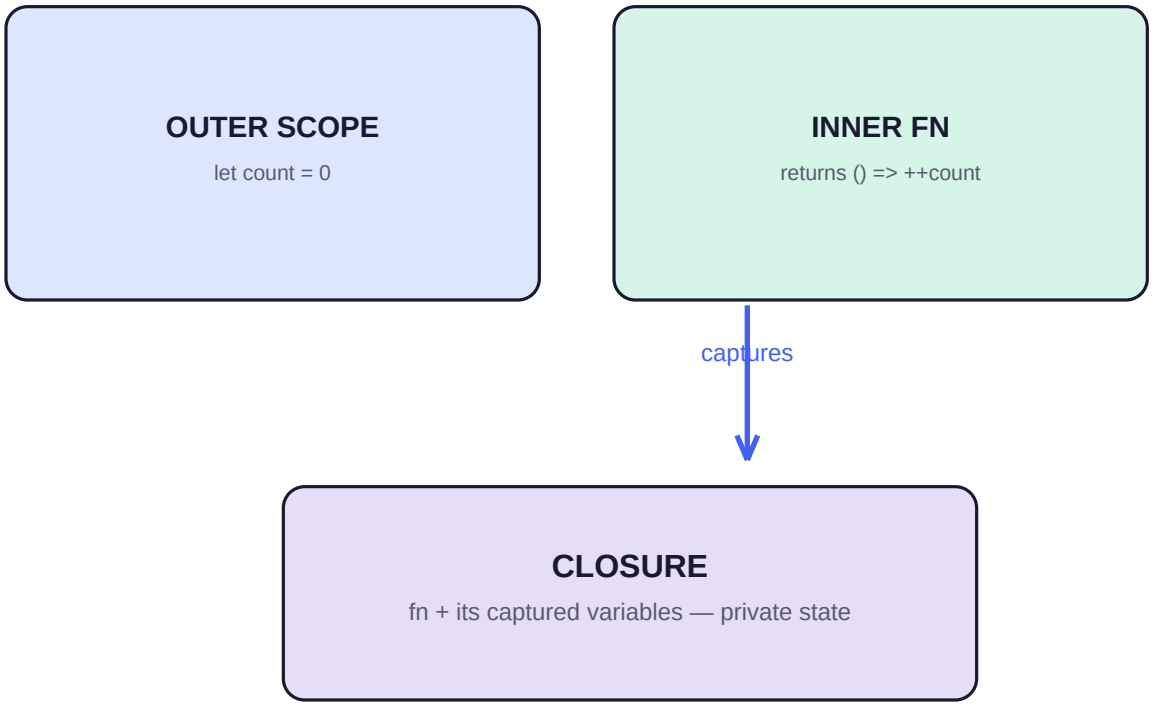
macrotask -> ALL its microtasks -> render? -> next macrotask

Promise chains starving the loop: microtasks can delay rendering indefinitely. Yield with setTimeout(0).

STEP 4/6

Closures: Functions Remember Scope

A function carries its birth environment wherever it goes.

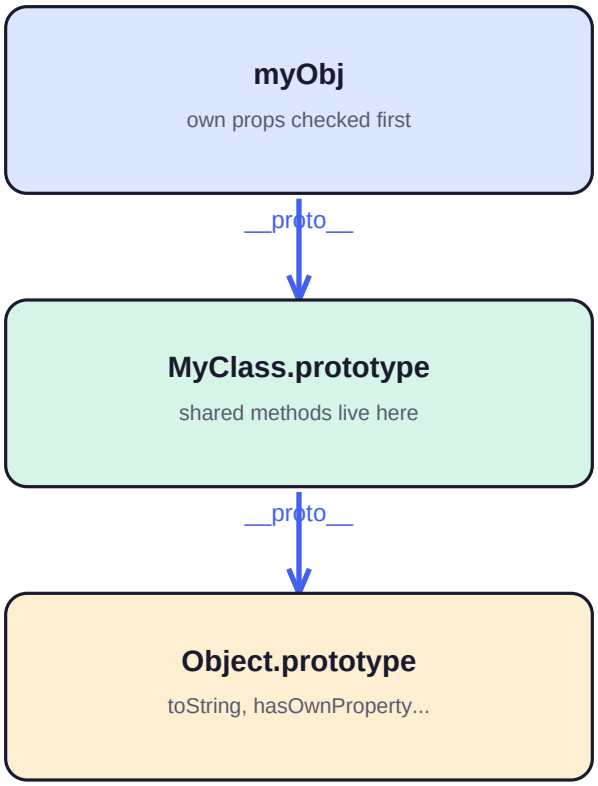


Modules, callbacks, memoization, React hooks: all closures. Watch for accidental retention -> memory leaks.

STEP 5/6

The Prototype Chain

Property lookup walks up: object -> prototype -> ... -> null.



class syntax is sugar over this chain. Methods on the prototype are shared, not copied.

STEP 6/6

async/await: Syntax Over the Loop

No new threads — just promises scheduled on the queues you already know.

async function

pauses at await; resumes as a microtask when the promise settles

SEQUENTIAL

await a; await b — b waits for a

PARALLEL

Promise.all([a, b]) — both in flight

await in a loop is the classic perf bug: independent calls should be Promise.all'd.

STEP ✓I✓

Key Takeaways

JS Runtime — Remember

- One thread, one call stack: long sync work blocks rendering and input.
- The event loop pumps tasks from the queue whenever the stack empties.
- Microtasks (promises) drain fully before the next macrotask (timers, I/O).
- Closures bundle a function with its birth scope — private state and the basis of modules.
- Property lookup walks the prototype chain; class is sugar over prototypes.
- async/await schedules on the same queues — use Promise.all for independent work.

Keep exploring

Pair this guide with the matching video walkthroughs
and hands-on labs at interstitiumlabs.dev